

Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of March 17, 2026

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

Abstract	ii
1. Introduction	1
1.1. Motivation	1
1.2. Predecessor Project: “Project 2”	1
1.3. Project Goals	1
1.3.1. Research Goals	2
1.3.2. Framework	3
1.3.3. Scenarios	5
1.3.4. Evaluation Goals	7
1.3.5. Out of Scope / Optional Feature	8
2. Related Research	9
2.1. Framework Comparison	9
2.1.1. Atomic Red Team	10
2.1.2. Caldera	11
2.1.3. Metasploit	11
2.1.4. NSAK	11
3. Concepts	13
4. Methodology	14
5. Implementation	15
5.1. Configuration Management Implementation	16
5.1.1. Device Configuration Management	16
5.1.2. Drill Configuration Management	20
5.1.3. Scenario Configuration Management	20
5.1.4. Configuration via CLI	20
5.1.5. Configuration via CLI -config_file	20
5.1.6. Configuration via interactive CLI Dialog	20
6. Evaluation	21
7. Discussion	22
8. Conclusion	23

Bibliography	24
List of Figures	25
List of Tables	26
Listings	27
Glossary	28
A. Project Management	29
A.1. Stakeholders	29
A.2. Agile Management Process	29
A.2.1. Issue Lifecycle:	30
A.2.2. Definition of Ready (DoR)	30
A.2.3. Definition of Done (DoD)	31
A.3. Project Structure	31
A.3.1. Milestones	31
A.3.2. Epics	32
A.3.3. Issues	32
A.4. Sprints / Iterations	32
A.4.1. Planning	32
A.5. Planning and Estimation	32
A.6. Project Roadmap	32
A.7. Sprint Ceremonies	34
A.7.1. Sprint1	34
A.7.2. Sprint2	36
A.7.3. Sprint3	38
A.7.4. Sprint4	38
A.7.5. Sprint5	38
A.7.6. Sprint6	38
A.7.7. Sprint7	38
A.7.8. Sprint8	38
A.7.9. Risk Assessment	39
A.8. Meeting Notes	44
A.9. Variant Decision: Thesis Goals	47
A.9.1. Variant 1: AI vs Manually Built Scenarios (Pivot)	47
A.9.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)	48
A.9.3. Decision	48
A.10. Variant Decision: AI Integration	49
A.10.1. Variant 1: AI Agent in NSAK	49
A.10.2. Variant 2: NSAK MCP Server	49
A.10.3. Decision	49

A.11. Workshops	50
A.11.1. Brainstorm Scenario Selection	50
A.11.2. NSAK Scenario: Network Discovery - Red Team	52
A.11.3. NSAK Scenario: TLS Downgrade Poodle - Red Team	53
A.11.4. NSAK Scenario: Honeypot - Blue Team	56
A.11.5. NSAK Scenario: Traffic Capture - Red Team	56
A.11.6. NSAK Scenario: Intrusion Detection - Blue Team	57
A.12. Self-Hosted LLM	59
A.12.1. Hardware Setup	59
A.12.2. System Setup	59
A.12.3. Evaluation	65

1. Introduction

Computer networks are constantly exposed to evolving cyber threats ...

The *Network Swiss Army Knife (NSAK)* is a framework designed to execute orchestrated security scenarios consisting of reusable attack drills in a controlled network environment. The framework aims to support automated security testing and adversary emulation on network edge devices.

The development of the NSAK framework originates from previous project work, in which the core architecture and the basic scenario execution model were developed. This thesis builds upon this foundation and extends the framework by investigating how AI agents can support automated red and blue team scenarios.

...

1.1. Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2. Predecessor Project: “Project 2”

1.3. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- ▶ **Specific:** What do we want to achieve?
- ▶ **Measurable:** What are the success criteria?
- ▶ **Achievable:** How do we plan to achieve this goal?
- ▶ **Relevant:** Why is it important to achieve this goal?

Describe what we did in project 2 and what project 2 is (BFH module, possible predecessor to bachelor thesis)

- ▶ **Time-bound:** Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.3.1. Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- ▶ Concepts
- ▶ Modularity
- ▶ Configurability
- ▶ Automation
- ▶ AI Capabilities

Success criteria:

- ▶ At least two security emulation frameworks are identified and reviewed
- ▶ A matrix comparing the specified properties of the selected frameworks is created
- ▶ The USP of the NSAK framework is identified

AI Integration (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- ▶ How is AI currently used in cybersecurity automation?
- ▶ Which tasks in adversary emulation can be supported or automated by AI?
- ▶ How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- ▶ What architectural changes would be required to integrate AI into NSAK?

1.3.2. Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- ▶ Scenarios and drills can expose configurable parameters
- ▶ The available parameters can be listed by the CLI with the `--help` option
- ▶ Parameters can be provided via CLI arguments or YAML files
- ▶ The framework resolves the inputs into a run configuration (internal data structure).

REST API (TBD)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal [1.3.1](#) and on whether we want to fulfill the goal [1.3.2](#), which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- ▶ A REST API specification with feature parity to the CLI is created.

- ▶ The REST API is implemented according to the specification.
- ▶ The API is documented (Open API) and usable by external clients.
- ▶ The REST API remains optional and does not replace the CLI-based interaction.
- ▶ Authentication and Authorization are explicitly out of scope.

MCP Server (TBD)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.3.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- ▶ A MCP-Server is specified and implemented
- ▶ Documentation how to connect to the MCP is provided.
- ▶ A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- ▶ A simple Web GUI that interacts with the REST API has been implemented.
- ▶ The Web GUI displays a list of available scenarios and their states.
- ▶ The scenario's results can be viewed on the Web GUI
- ▶ Scenarios can be configured and executed via the Web GUI
- ▶ Authentication and authorization are explicitly out of scope.

1.3.3. Scenarios

AI – Purple Team (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- ▶ The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- ▶ The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- ▶ An operator can interact with or stop the running AI.
- ▶ The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- ▶ The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- ▶ NSAK scenarios and drills can be executed consistently within the environment.
- ▶ The environment supports both manual and AI-assisted scenarios.
- ▶ Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- ▶ A red team network discovery scenario is implemented in the NSAK framework.
- ▶ Discovered network information is collected as artifacts.
- ▶ The scenario can be executed reproducibly within the test environment.
- ▶ The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- ▶ A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- ▶ Detection events or alerts can be observed and documented during scenario execution.
- ▶ Detection results can be compared with results from AI-assisted scenario execution.

1.3.4. Evaluation Goals

AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.3.3 within the network environment defined by the goal 1.3.3. The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- ▶ A methodical approach for the assessment is evaluated and documented.
- ▶ The results of the AI scenario for red and blue team activity are assessed.
- ▶ There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- ▶ A methodical approach for the comparison is evaluated and documented.
- ▶ The results of the AI scenario are compared against the respective results of the red team scenario (must)
- ▶ The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- ▶ There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

1.3.5. Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table ?? in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

2. Related Research

2.1. Framework Comparison

Previous research, such as ‘SOK: A Survey of Open-Source Threat Emulators’, compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [1]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit recieved high scores in categories as environments and operator, environment configuration and scenario execution.

n the following section, the selected frameworks are briefly introduced, and the unique selling point of the NSAK is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [2, 3].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [2]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [4–6] and the findings of Zilberman et al. [1].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium-High	Low-Medium	Medium-High

Table 2.1.: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([1, 4–6])

As shown in Table ??, the key characteristics of the frameworks are differ greatly. For **Atomic Red Team** provides libraries of attack scripts that implement individual TTP of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

2.1.1. Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information. [5]

The framework can be executed directly from the command line and provides

features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites. [5]

Zilberman et al. recommends deeper cyber security knowledge to use Atomic Red Team framework effectively [1]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2. Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the TTP. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional feature, like a training plugin for educational purposes, includes facts that can be configured as dynamic safe values, which can be stored and parsed in later steps. [4]

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and CNC communication [1].

2.1.3. Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the CLI requires knowledge of the target network and system and requires partial configuration. [6, 7]

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [1].

2.1.4. NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks.

An approach could be to use an AI-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead. [8]

AI in framework

3. Concepts

4. Methodology

5. Implementation

5.1. Configuration Management Implementation

5.1.1. Device Configuration Management

Create New Device Resource Type

In the context of “Project 2”[1.2](#) this resource type was already defined conceptionally and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r4/`. But the resource type was not actually added to the NSAK core and be able to manage device configurations we need to implement the Device resource type first. Analog to the other resource types, a yaml specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. The listing [5.1](#) shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

```

1 device: <str:resource-version>
2
3 metadata:
4   id: <str:device-id>
5   name: <str:device-name>
6   description: <str:device-description>
7   author: <str:author-email>
8   repository: <str:resource-repository>

```

Listing 5.1: Device Resource YAML Specification

In the following table [5.1](#) the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

Class	File Name	Description
Device	device.py	A dataclass, which represents a device resource as objects during runtime (usually quite similar to the yaml specification).
DeviceLoader	device_loader.py	This class returns Device objects and is used by the DeviceManager to find and load device resources from the library.
DeviceManager	device_manager.py	The manager class implements the business logic and exposes an API, which can be used by the CLI.

Table 5.1.: Device Resource Classes

In “Project 2”[1.2](#) it became already clear, that the described classes share a lot of boilerplate code between each other. For this reason the base classes described

in 5.1 where added to get rid of the duplicated logic, which leads to better compliance with the Don't Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files.

Class	File Name	Description
Resource	<code>resource.py</code>	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	<code>resource_loader.py</code>	Base class which handles searching and loading resources from the filesystem.
ResourceManager	<code>resource_manager.py</code>	Base class which defines common methods for all resource manager classes.

Table 5.2.: Resource Python Classes

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing 5.2.

```

1  # Show usage, options and subcommands for the device resource
2  nsak device --list
3
4  # List all devices found in the resource library
5  nsak device list
6
7  # Output:
8  nsak device list
9  ID          NAME          PATH
10 bananapi_r4  BananaPI R4
   /home/lukas/CLionProjects/swiss-army-knife-network-sniffer/lib/devices/bananapi_r4

```

Listing 5.2: NSAK CLI: List Devices

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration as this is the most important information for the scenarios and drills, but the specification was done in such a way that other sections can be added later on. The following excerpt 5.3 shows the specification for the configuration section of the device resource YAML.

```

1  device: <str:resource-version>
2
3  metadata:
4    ...
5
6  configuration:

```

```

7  network:
8    ethernet:
9      <str:interface-name>:
10     addresses:
11       <str:ip-address-with-subnet>:
12         is_target: <boolean>
13         is_management: <boolean>

```

Listing 5.3: Device Resource Configuration YAML Specification

The following code snippet located under `src/nsak/core/device/device_loader.py` 5.4 shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/device/device.py` 5.5.

```

1  class DeviceLoader(ResourceLoader[Device]):
2      """
3      Finds and loads devices from the library paths.
4      """
5
6      ResourceClass = Device
7
8      @classmethod
9      def _parse_device_configuration(
10         cls, data: dict[str, Any]
11     ) -> DeviceConfiguration | None:
12         if "configuration" not in data:
13             return None
14
15         ethernet = data["configuration"].get("network", {}).get("ethernet", {})
16
17         return DeviceConfiguration(
18             network=NetworkConfiguration(
19                 ethernet={
20                     interface: EthernetConfiguration(
21                         name=interface,
22                         addresses={
23                             ip: IPConfiguration(
24                                 ip=ip_interface(ip),
25                                 is_target=address.get("is_target", False),
26                                 is_management=address.get("is_management", False),
27                             )
28                         for ip, address in ethernet.get("addresses", {}).items()
29                     },
30                 )
31             for interface, ethernet in ethernet.items()
32         )
33     )
34
35     @classmethod
36     def _to_resource(cls, data: dict[str, Any], path: Path) -> Device:
37         """
38         Creates a Device object from a dict containing the device's metadata.
39         """
40
41         return cls.ResourceClass(
42             id=str(data["metadata"]["id"]),
43             name=str(data["metadata"]["name"]),
44             path=path,
45             author=str(data["metadata"]["author"]),
46             repository=str(data["metadata"]["repository"]),
47             configuration=cls._parse_device_configuration(data),

```

48)

Listing 5.4: DeviceLoader Configuration Parsing

```

1  @dataclasses.dataclass(frozen=True, kw_only=True)
2  class IPConfiguration:
3      """
4      Represents an ip configuration on an interface.
5      """
6
7      ip: IPInterface
8      is_target: bool
9      is_management: bool
10
11
12  @dataclasses.dataclass(frozen=True, kw_only=True)
13  class EthernetConfiguration:
14      """
15      Represents an ethernet node.
16      """
17
18      name: str
19      addresses: dict[str, IPConfiguration]
20
21
22  @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
23  class NetworkConfiguration:
24      """
25      Represents the networking part of the configuration, heavily inspired by netplan.
26      """
27
28      ethernets: dict[str, EthernetConfiguration]
29
30
31  @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
32  class DeviceConfiguration:
33      """
34      Represents the device configuration.
35      """
36
37      network: NetworkConfiguration
38
39
40  @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
41  class Device(Resource):
42      """
43      Represents a device.
44      """
45
46      id: str
47      name: str
48      path: Path
49      author: str
50      repository: str
51      configuration: DeviceConfiguration | None = None

```

Listing 5.5: Device Configuration Datastructures

In the following listing shows the newly available CLI commands for managing device configurations ??:

```
1  # List all subcommands for the Device resource  
2  nsak device --help  
3  
4  # List all available devices  
5  nsak device list  
6  
7  # Show the device details and configuration  
8  nsak device show <str:device-id>  
9  
10 # Load a device and its configuration  
11 nsak device load <str:device-id>  
12  
13 # Show the currently loaded device  
14 nsak device loaded  
15  
16 # Reset the loaded device  
17 nsak device unload
```

Listing 5.6: Device Configuration CLI Management

5.1.2. Drill Configuration Management

5.1.3. Scenario Configuration Management

5.1.4. Configuration via CLI

5.1.5. Configuration via CLI `-config_file`

5.1.6. Configuration via interactive CLI Dialog

6. Evaluation

7. Discussion

8. Conclusion

Bibliography

- [1] Polina Zilberman, Rami Puzis, Sunders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. arXiv preprint, 2020.
- [2] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.
- [3] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.
- [4] MITRE Corporation. Mitre caldera documentation, 2024.
- [5] Red Canary. Atomic red team, 2024.
- [6] Rapid7. Metasploit framework documentation, 2024.
- [7] David Kennedy, Jim O’Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester’s Guide*. No Starch Press, 2011.
- [8] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). <https://github.com/nsak-team/nsak>, 2026. Accessed: 2026-01-10.
- [9] Advisera. Iso 27001 risk assessment, treatment, and management: The complete guide, 2025. Accessed: 2026-03-04.
- [10] Simplr. General recommended vram guidelines for llms, 2025. Online; accessed 10 March 2026.

List of Figures

A.1. Roadmap	33
A.2. Checklist and Burndown Sprint1	34
A.3. Retro Sprint 1	35
A.4. Risks Brainstorming	41
A.5. Risks Brainstorming	42
A.6. Risk Assessment	43
A.7. Variant 1: AI vs Manually Built Scenarios	47
A.8. Variant 2: Multiple Blue and Red Team Scenarios	48
A.9. AI Integration Variants	49
A.10. Brainstorm Scenarios	50
A.11. Clustering possible Scenarios	51
A.12. NSAK Scenario: Network Discovery - Red Team	53
A.13. NSAK Scenario: TLS Downgrade Poodle - Red Team	54
A.14. NSAK Scenario: TLS Downgrade Poodle -Topology	55
A.15. NSAK Scenario: Honeypot - Blue Team	56
A.16. NSAK Scenario: Traffic Capture - Red Team	57
A.17. NSAK Scenario: Intrusion Detection - Blue Team	58
A.18. FRITZ!Box DynDNS Setup	64
A.19. Infomaniak DynDNS Setup	64
A.20. FRITZ!Box DynDNS Setup	65
A.21. FRITZ!Box DynDNS Setup	66

List of Tables

2.1. Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([1, 4–6])	10
5.1. Device Resource Classes	16
5.2. Resource Python Classes	17
A.1. Key Scrum Meetings	29
A.2. Issue Board	30
A.3. Definition of Ready Criteria for Tickets	30
A.4. Definition of Done Criteria for Tickets	31
A.5. Project Milestones in GitLab	31
A.6. Planned Sprint Activities	36
A.7. Risk categories used for the project risk assessment	39
A.8. Risk evaluation criteria	39
A.9. Risk probability scale	40
A.10. Risk impact scale	40
A.11. Risk level classification (probability and impact are interchangeable)	40
A.12. Risk Treatment Strategies	42
A.13. Meeting Notes: Checkpoint 1	44
A.14. Meeting Notes: Checkpoint 2	45
A.15. Hardware configuration of the self-hosted LLM inference server.	59

Listings

5.1. Device Resource YAML Specification	16
5.2. NSAK CLI: List Devices	17
5.3. Device Resource Configuration YAML Specification	17
5.4. DeviceLoader Configuration Parsing	18
5.5. Device Configuration Datastructures	19
A.1. Local LLM System Setup	60
A.2. /home/ai/docker-compose.yml	61
A.3. swag/nginx/site-confs/default.conf	62
A.4. UFW configuration	63
A.5. Restart SWAG Container	65

Glossary

This document is incomplete. The external file associated with the glossary ‘main’ (which should be called `thesis.gls`) hasn’t been created.

Check the contents of the file `thesis.gls`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

If you don’t want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- ▶ Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- ▶ Run the external (Lua) application:

```
makeglossaries-lite.lua "thesis"
```

- ▶ Run the external (Perl) application:

```
makeglossaries "thesis"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

A. Project Management

A.1. Stakeholders

Hj. Wenger: Instructor and Network Specialist

Urs Keller: Expert and Entrepreneur

A.2. Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

Sprint Planning	Sprint Review	Sprint Retro	Checkpoint
Select milestone with sprint goal	Close open tickets	Reflect on sprint	Present sprint results
Check issues for DoR and obstacles	Gather stakeholder feedback	Identify improvements	Get stakeholder feedback
Ensure issues are clear	Validate delivered features	Discuss what went well	Define next tasks
Create sprint backlog	Review progress	Discuss issues/problems	Discuss issues/problems

Table A.1.: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

A.2.1. Issue Lifecycle:

Buckets	Description
Backlog	List of all tasks and features planned for the project.
Todo	Tasks selected for the current sprint that are ready to be worked on.
Q&A	Tasks waiting for clarification, review, or testing.
Done	Completed tasks that meet the defined acceptance criteria.

Table A.2.: Issue Board

A.2.2. Definition of Ready (DoR)

Category	Criteria
Clarity	▶ Clear and precise title ▶ Objective or problem described in two to three sentences ▶ Scope clearly defined
Scope	▶ Affected module specified (drill, scenario, core, container, frontend) ▶ External dependencies identified
Validation	▶ Testable acceptance criteria defined ▶ Expected behavior clearly specified

Table A.3.: Definition of Ready Criteria for Tickets

A.2.3. Definition of Done (DoD)

Category	Criteria
Implementation	▶ All acceptance criteria fulfilled ▶ Container execution verified ▶ Pre-commit hooks pass ▶ Logging and error handling consistent
Testing	▶ Feature or bug fix tested ▶ Acceptance criteria verifiable (automated or manual) ▶ Relevant edge cases considered ▶ Execution reproducible
Documentation	▶ README updated if required ▶ Architecture changes documented ▶ Thesis relevance briefly noted ▶ Limitations documented ▶ Required documentation stored for thesis or appendix

Table A.4.: Definition of Done Criteria for Tickets

A.3. Project Structure

A.3.1. Milestones

A milestone represent crucial steps toward the overall project goal. Per our definition a milestone has at least one epic and the mandatory milestones are listed in [Table A.5](#)

GitLab Milestones
Milestone Project Management
Research AI Integration
Research: Compare Frameworks
NSAK Framework: Configuration Management
NSAK Scenario: Network Discovery - Red Team
NSAK Scenario: AI - Purple Team
Reproducible Test Environment

Table A.5.: Project Milestones in GitLab

A.3.2. Epics

consist of multiple stories and contribute to fulfilling milestones.

A.3.3. Issues

Each Story is assigned a weight, sprint, milestone and epic and are usually represented by an “Issue” in gitlab.

A.4. Sprints / Iterations

A.4.1. Planning

“Task” and “Issue”, the terms are often used interchangeably with “Stories”. Stories must meet the [A.2.2](#) before implementation and [A.2.3](#) before closing.

A.5. Planning and Estimation

The following estimation strategy was used:

- ▶ 1 hour corresponds roughly to 1 weight point
- ▶ Iteration length: 2 weeks
- ▶ Weights per iteration: 40h per person = 80 weight per iteration
- ▶ Sprint planning duration: 1.5 hours per iteration
- ▶ Sprint review duration: 1 h per iteration
- ▶ Sprint retro duration: 1 h per iteration

A.6. Project Roadmap

The introduced project structure is illustrated in the roadmap shown in [A.1](#). The structure shown in [A.3](#) helps to visualize the project goals and keeps track of the progress towards achieving goals and milestones. Furthermore, the visualization helps stakeholders to grasp the current state of the project.

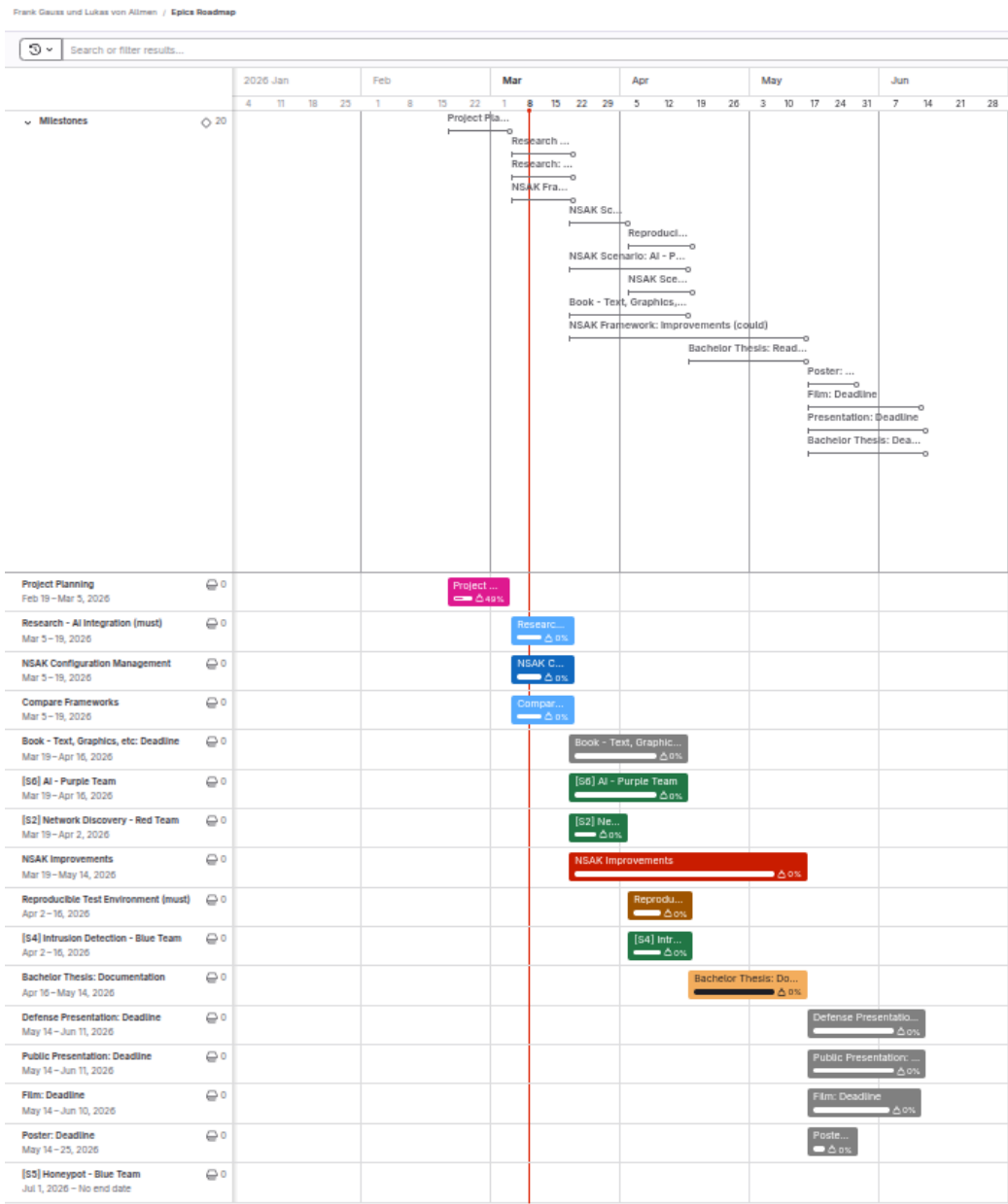


Figure A.1.: Roadmap

A.7. Sprint Ceremonies

A.7.1. Sprint1

Planning

In planning of sprint1 we derived all issues from the milestone project management A.5 and had an overall weight from 89

Review

All deliverables were fulfilled. However, the deadlines of the milestone had to be adjusted because project management required more time than initially expected. The milestone was successfully reached, and the board was cleared in preparation for the next sprint. All issues were reviewed and moved to the *Done* column.

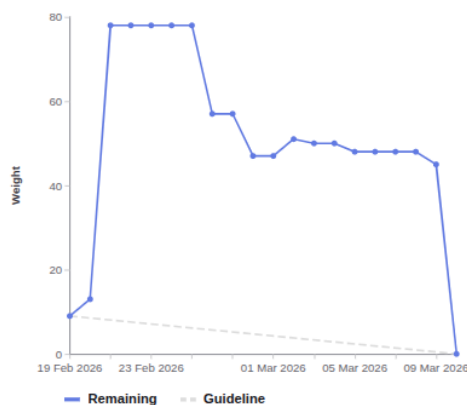
Feedback and demonstrations were not considered necessary during the Sprint Review. Instead, they were moved to dedicated checkpoints together with the stakeholders.

Project Planning

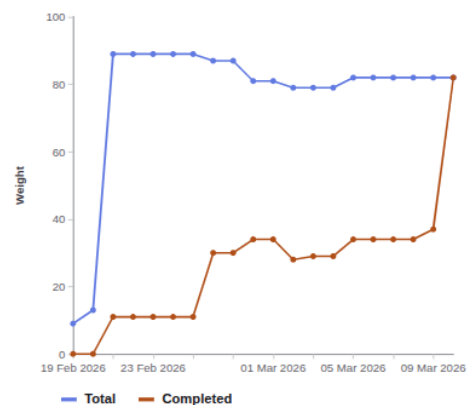
- ☒ The bachelor thesis is planned as an agile (scrum like) project
- ☒ A risk analysis for the project was conducted
- ☒ The identified risks are assessed and addressed
- ☒ All meetings, deadlines, scrum ceremonies and sprints are added to the outlook calendar
- ☒ The goals of the thesis are defined and categorized as "must", "should" and "can"
- ☒ The goals are created as milestones and divided into epics, which are categorized by the labels: "Project Management", "Research", "Framework", "Implementations", "Documentation"
- ☒ We know how we coordinate the project with the stake holders (instructor and expert)

Display by Count Weight

Burndown chart



Burnup chart



A.7.2. Sprint2

Planning

Week	Responsible	Task
Week 1	Lukas	Configuration management implementation
Week 1	Frank	Research and comparison of frameworks
Week 1	Team	AI workshop on Thursday and writing of the related issues
Week 2	Team	AI research and evaluation
Week 2	Team	Preparation and pitch of the project deliverables

Table A.6.: Planned Sprint Activities

Review

Retro

A.7.3. Sprint3

Planning

Review

Retro

A.7.4. Sprint4

Planning

Review

Retro

A.7.5. Sprint5

Planning

Review

Retro

A.7.6. Sprint6

Planning

Review

Retro

A.7.7. Sprint7

Planning

Review

Retro

A.7.8. Sprint8

38

Planning

Review

Retro

A.7.9. Risk Assessment

To methodically conduct the risk assessment the section “Risk assessment” from an online guide published by Advisera [9], which describes the risk assessment process according to ISO 27001, was used as a reference.

Risk Assessment Process:

1. Risk categories and criteria: Define which dimensions and taxonomies are important for the assessment.
2. Risk identification: Collect possible risks in a brainstorming session.
3. Risk analysis: Assess the risks criteria on a risk matrix.
4. Risk evaluation: Create a list of risks ordered by the resulting risk level and evaluate if they are acceptable or need treatment.
5. Risk treatment: Describe the strategy and how to treat the risks.

Risk Categories & Criteria:

The table A.7 shows four commonly used categories, which were used to group the risks.

Category	Description
Strategic Risk	Risks related to business decisions (e.g., wrong prioritization).
Operational Risk	Risks related to internal processes (e.g., Scrum, QA process).
Financial Risk	Risks related to costs and expenses (e.g., hardware costs).
External Risk	Risks related to uncontrollable sources (e.g., force majeure).

Table A.7.: Risk categories used for the project risk assessment

The criteria for analysing the risks are summarized in the table A.8

Criterion	Description
Probability	Likelihood that the risk will occur.
Impact	Severity of consequences if the risk materializes.
Level	Risk level calculated as Probability + Impact.

Table A.8.: Risk evaluation criteria

The levels of the criteria and their respective numerical weights are listed in the tables A.9 and A.10.

Level	Weight	Description
Low	1	Unlikely
Medium	2	Possible
High	3	Likely

Table A.9.: Risk probability scale

Level	Weight	Description
Low	1	Minor inconveniences
Medium	2	Delays milestones or reduces scope
High	3	Threat to the thesis

Table A.10.: Risk impact scale

The following table [A.11](#) defines the calculation method and the resulting risk levels, used for the assessment.

Risk Level	Weight	Calculation
Very Low	2	Low + Low
Low	3	Low + Medium
Medium	4	Medium + Medium / Low + High
High	5	Medium + High
Critical	6	High + High

Table A.11.: Risk level classification (probability and impact are interchangeable)

Risk Identification

The identification process was conducted with the help of a digital brainstorming, where the risks are already grouped by category. The identified risks are represented by the cards shown in [A.4](#). The respective risk categories are visualized by the color coding of the cards and described in the legend.

Risk Brainstorming

- **Strategic Risk:** Risks related to business decisions (e.g. wrong prioritization).
- **Operational Risk:** Risks related to internal processes/procedures (e.g. Scrum, QA process).
- **Financial Risk:** Risks related to costs and expenses (e.g. hardware costs).
- **External Risk:** Risks related to uncontrollable sources (e.g. Force majeure).

An existing framework already does everything we have planned	Not enough time for testing (drills, scenarios and environments)	Not enough time for media artifacts (Poster, Presentations, Film, Book, etc.)	Lack of resources/ consensus for defense scenarios	Complex Network environments need more hardware than available	One of the students gets injured or sick
Misaligned communication lead to wrong expectations	Not enough time for documentation	Drills not compatible between scenarios	Lack of resources for attack scenarios	AI Credits are too expensive to build and test the AI scenario	NSAK gets abused by hackers
Scope is too large for bachelor thesis	Scenarios and drills need more time as planned	Regressions: Tested scenarios stop working, because a drill was changed	User do not understand how to operate NSAK	The NSAK hardware devices stop working	The AI bubble bursts and we need to switch AI provider
Scenario or drill too complex for automation	Simulate or build network environments need more time as planned	The NSAK framework/ device itself is vulnerable	The AI might go rough in the AI scenario		

Figure A.4.: Risks Brainstorming

Risk Analysis

For analysing the risks a matrix was prepared, where the criteria “Impact” and “Probability” are representing the x- and y-axis respectively. The project team then assessed the criteria for each individual risk and placed them on the matrix accordingly. In a second step, the risks where moved if they did not fit relatively to each other. The resulting “Risk Analysis Matrix” is illustrated in the following figure [A.5](#). The background color of the cells is representing the resulting risk level of the tasks.

Risk Matrix

Probability	High	The NSAK framework/device itself is vulnerable User do not understand how to operate NSAK Regressions: Tested scenarios stop working, because a drill was changed Not enough time for testing (drills, scenarios and environments) Simulate or build network environments need more time as planned AI Credits are to expensive to build and test the AI scenario
	Medium	An existing framework already does everything we have planned The AI might go rough in the AI scenario Complex Network environments need more hardware than available Scenarios and drills need more time as planned Not enough time for documentation Scope is too large for bachelor thesis Drills not compatible between scenarios Misaligned communication lead to wrong expectations The AI bubble bursts and we need to switch AI provider Not enough time for media artifacts (Poster, Presentations, Film, Book, etc.) Scenario or drill to complex for automation
	Low	Lack of resources for attack scenarios Lack of resources/consensus for defense scenarios The NSAK hardware devices stop working NSAK gets abused by hackers One of the students gets injured or sick
		Low Medium High
		Impact

Figure A.5.: Risks Brainstorming

Risk Evaluation & Treatment

In a first step a table with the required columns was created. Then the risks and weights for the criteria were added with an ID in the format “R-000”, to easier reference them in the thesis later on. After sorting the table by the calculated risk level, the risk evaluation was conducted by deciding which treatment strategy described in A.12 we want to apply.

Strategy	Description
Mitigate	Reduce the likelihood or impact of a risk.
Transfer	Move the risk to a third party.
Accept	Acknowledge the risk and take no further action.
Avoid	Eliminate or circumvent the cause of the risk.

Table A.12.: Risk Treatment Strategies

Finally, the project team went through the risks and discussed how to treat them effectively. The resulting “Risk Assessment” is illustrated in the following table A.6.

Figure A.6.: Risk Assessment

Risk Assessment									
Risk ID	Description	Risk Category			Severity			Criticality	
		Financial	Strategic	Probability	Low	Medium	High	Criticality	
								Operational	Reputational
R001	AI models are too expensive to build and test the AI scenario	Financial	3	3	9	3	9	Medium	High
R006	Scalp is too large to butcher tickets	Strategic	2	3	6	2	6	Medium	High
R007	Scenario or data too complex to automate	Strategic	2	3	6	2	6	Medium	High
R002	Not enough time for documentation	Operational	2	3	6	2	6	Medium	High
R003	Not enough time for testing (defects, scenarios and environments)	Operational	3	3	9	3	9	Medium	High
R004	Simulating or building network environments needs more time than planned	Operational	3	3	9	3	9	Medium	High
R005	User does not understand how to operate NSMx	Operational	3	2	6	3	6	Medium	High
R012	Misaligned communication leads to wrong expectations	Strategic	2	2	4	2	4	Medium	High
R010	Scenarios and data need more time than planned	Operational	2	2	4	2	4	Medium	High
R011	Not enough time for media artifacts (goats, presentation, film, book, etc.)	Operational	2	2	4	2	4	Medium	High
R008	Complex network environments need more hardware than available	Financial	2	2	4	2	4	Medium	High
R006	The Alibaba bursts and are need to switch AI provider	External	3	2	6	3	6	Medium	High
R014	Regression tested scenarios stop working because a drift was changed	Operational	2	1	2	2	4	Medium	High
R015	The NSMx framework/device itself is vulnerable	Operational	3	1	3	3	3	Medium	High
R013	One of the students gets injured or sick	External	1	3	3	1	3	Low	Medium
R019	An existing framework already does everything we have planned	Strategic	2	1	2	2	2	Low	Medium
R017	Drills not compatible between scenarios	Operational	2	1	2	2	2	Low	Medium
R018	The AI might go rogue in the AI scenario	Operational	2	1	2	2	2	Low	Medium
R016	The NSMx hardware devices stop working	Financial	1	2	2	1	2	Low	Medium
R021	Lack of resources for alibaba scenarios	Operational	1	1	1	1	1	Low	Medium
R022	Lack of resources / constraints for alibaba scenarios	Operational	1	1	1	1	1	Low	Medium
R020	NSMx gets abused by hackers	External	1	1	1	1	1	Low	Medium

A.8. Meeting Notes

Meeting Notes Nsak Checkpoint - 26.02.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Goals presentation via slides - Clarification whether there is a deadline or submission required for the presentation (besides adding it to the addendum). - Clarification whether the defense presentation should be an enhanced version of the public presentation or a separate, more technical presentation. - Proposal to establish a recurring meeting series every Thursday at 15:00 during the semester.
Tasks	- Hand in a set of success criteria for the project goals: We thought we can delivery the success criteria and the project goals as milestones in gitlab, we corrected this later on. - Conduct a risk assessment: A.7.9 - Contact expert via e-mail: Contacted at 21.02.26, response 03.03.26, invited to next checkpoint meeting 05.03.26, the expert takes part at the meeting every two weeks

Add slides if we can reproduce them

Table A.13.: Meeting Notes: Checkpoint 1

Meeting Notes Nsak Checkpoint - 05.03.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review of the project goals as milestones and project planning. - Review of risk assessment. - Discussion on the investigation and analysis of network communication between end devices and external networks (RED scenario: sniffing, filtering, and potential data exfiltration).
Decisions	- Variant 1 was selected for further development. - Open questions remain regarding the use and management of AI tokens.
Notes	- We misunderstood how to deliver the project goals and success criteria: Instead of defining the goals as milestone we added a goal section in the thesis. - Discussion on the rationale behind selecting Variant 1. - Consideration of whether a user interface is required when MCP functionality is available - Open Research Question. - NSAK integrates AI capabilities. - The development of a Web GUI is currently considered lower priority - suggestion of Urs Keller.
Tasks	- Send the slides of the final presentation and documentation of "Project 2" to Urs Keller (Expert). - Delivery the projects goals with a detailed description as part of the thesis by Saturday, 07.03.26: 1.3 - Finalize the project planning: A.6 - Write the first few chapters of the thesis:

Table A.14.: Meeting Notes: Checkpoint 2

Meeting Notes Nsak Checkpoint - 19.03.26

Meeting Notes Nsak Checkpoint - 02.04.26

Meeting Notes Nsak Checkpoint - 16.04.26

Meeting Notes Nsak Checkpoint - 30.04.26

Meeting Notes Nsak Checkpoint - 14.05.26

Meeting Notes Nsak Checkpoint - 28.05.26

Meeting Notes Nsak Checkpoint - 11.06.26

Variant 1: AI Purple Team

	Research	Framework	Scenarios (Red- and Blue-Team)	
Must	Framework Comparison	Configuration Management WEB GUI	AI Scenario Red Team AI Scenario Blue Team	Intrusion Detection Blue Team Network Discovery Red Team
Should		Framework Improvements		
Could	GAP Analysis			

Figure A.7.: Variant 1: AI vs Manually Built Scenarios

A.9. Variant Decision: Thesis Goals

During the first sprint of the project, which was focused on project management and planning, we identified the very promising hypothesis that red and blue team activities could be automated and/or enhanced by AI. Pursuing this hypothesis would change the projects focus and goals and thus requires the agreement and approval of the stakeholder. To communicate this circumstance efficiently and leave the option to go with the status quo, we prepared two options to conduct a “Variant Decision”. In the following two subsections the two variants presented in the second checkpoint meeting [A.14](#) are described and illustrated. It must be noted, that the variants and their visualizations [A.7](#) [A.8](#) are momentary snapshots and will not be updated for traceability reasons. For example, the Web GUI was already deprioritized during the discussion of the variants, which is reflected in the project goals [1.3](#) but not here.

A.9.1. Variant 1: AI vs Manually Built Scenarios (Pivot)

Figure [A.7](#) shows the project goals adjusted for the AI focused hypothesis, which is favored variant of the project team. In this option most of the evaluated scenarios will not be realized in favor of AI based scenarios, which also requires additional research [1.3.1](#). The basic idea is to implement scenarios for network discovery [1.3.3](#), intrusion detection [1.3.3](#) and an AI based scenario with the same capabilities [1.3.3](#). In the evaluation of the thesis we can then assess the quality of the AI based scenarios itself [1.3.4](#) and in comparison to the manually built scenarios [1.3.4](#).

Variant 2

	Research	Framework	Scenarios (Red- and Blue-Team)	
Must	Framework Comparison	Configuration Management WEB GUI	Traffic Capture and Exfiltration Red Team	Intrusion Detection Blue Team Network Discovery Red Team
Should		Framework Improvements	TLS Downgrade POODLE Red Team	
Could	GAP Analysis		Honeypot Blue Team	

Figure A.8.: Variant 2: Multiple Blue and Red Team Scenarios

A.9.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)

This is the fallback if the other option [A.9.1](#) is not approved by the stakeholders. Even if the project team does not favor this more conservative variant, it still represents a valid way to conduct an interesting project. As illustrated below [A.8](#), in this variant we propose to proceed with the previously planned scenarios for which we already conducted “User Story Mappings” documented under the workshops section [A.11](#). The focus would lie in more conventional scenarios, overall framework improvements and an in depth comparison with another framework in form of a GAP Analysis.

A.9.3. Decision

The stakeholders and project team decided in favor of **Variant 1: AI vs Manually Built Scenarios (Pivot)** [A.9.1](#) in the checkpoint meeting [A.14](#).

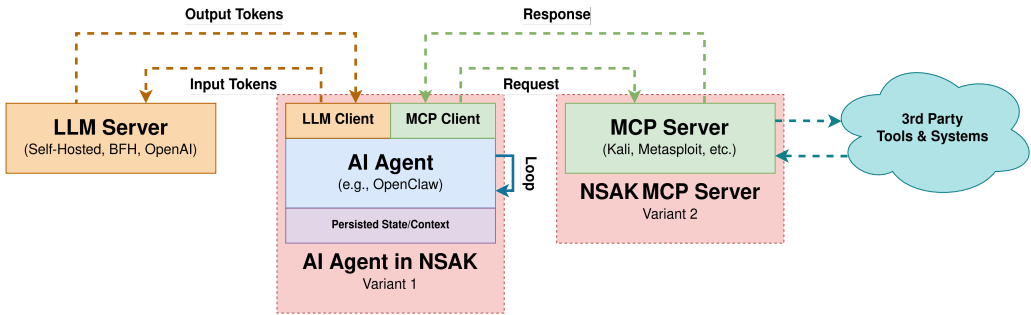


Figure A.9.: AI Integration Variants

A.10. Variant Decision: AI Integration

A.10.1. Variant 1: AI Agent in NSAK

A.10.2. Variant 2: NSAK MCP Server

A.10.3. Decision

Add the decision

A.11. Workshops

In the Project Management phase, several workshops were conducted, including a brainstorming session using Post-it notes and a User Story Mapping exercise on a whiteboard. The goal of these workshops was to clarify the scenarios a network edge device should support.

Following these workshops, the insights gathered were used to derive issues and goals, which were then mapped to detailed milestones in GitLab. All deliverables were categorized using the MoSCoW prioritization method into must, should, and could requirements. An example of this prioritization and scenario mapping is illustrated in Figure A.12. All Milestones links are provided in the URL at the top of the section.

A.11.1. Brainstorm Scenario Selection

In this workshop, participants brainstorm and cluster potential red and blue team scenarios for practical cybersecurity exercises.

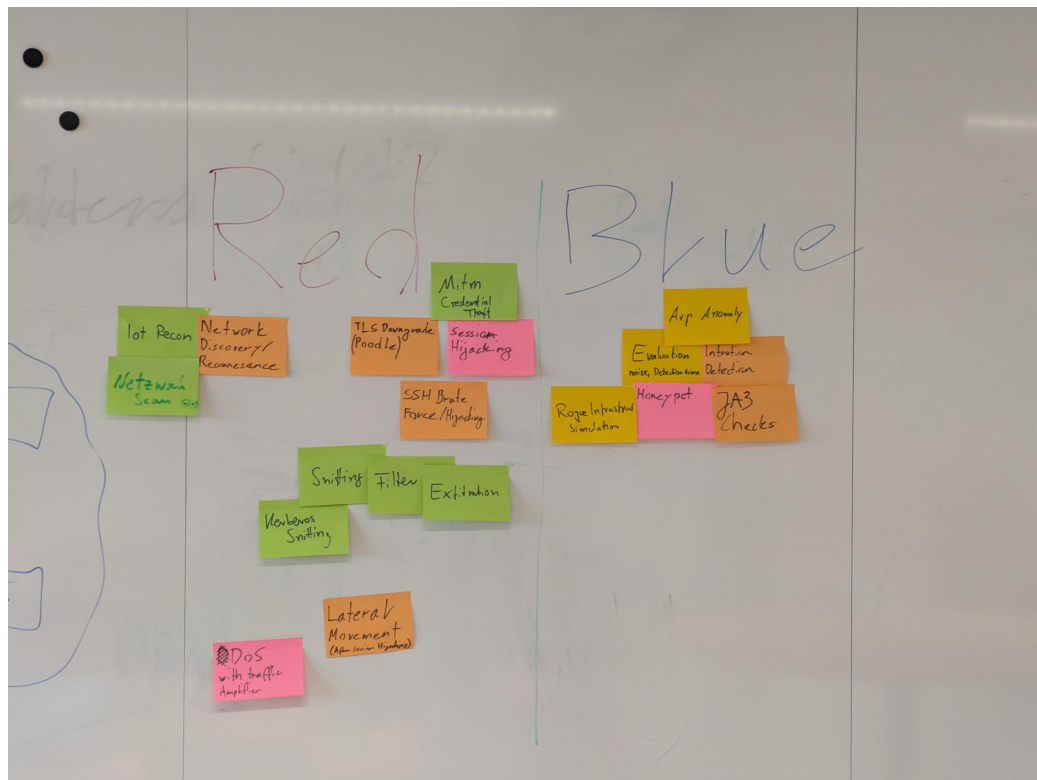


Figure A.10.: Brainstorm Scenarios

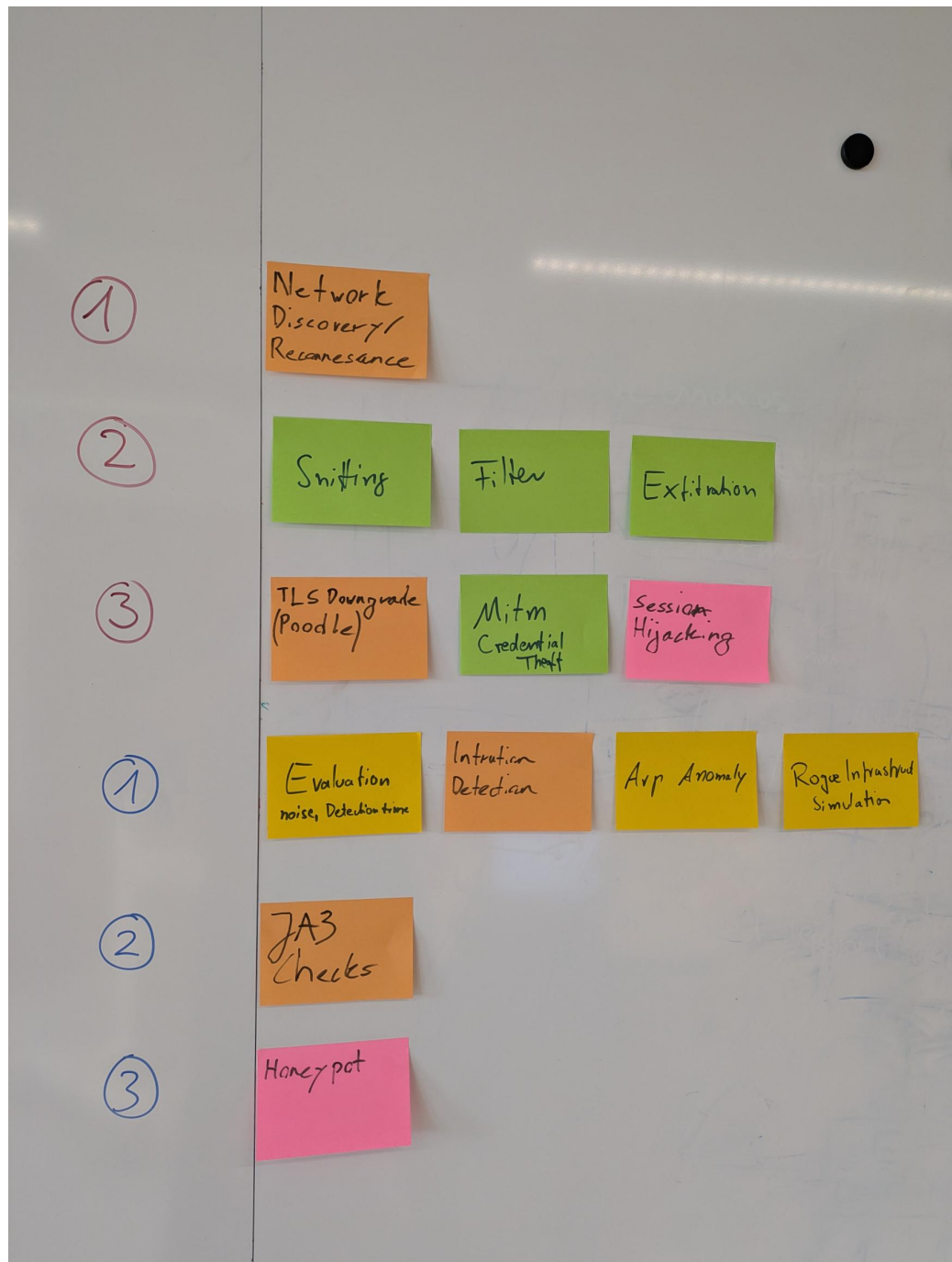


Figure A.11.: Clustering possible Scenarios

The Brainstorm session is used to derive further red/blue team scenarios and led to the pivot of AI-enabled Nsac scenario.

A.11.2. NSAK Scenario: Network Discovery - Red Team

Network Discovery Scenario

Objectives

This workshop aims to evaluate the different steps of the network discovery process and to help participants understand how attackers identify and map potential targets within a network.

Building on this objective, Red Teams consider network discovery one of the most essential activities during the reconnaissance phase, as it helps them identify hosts and services and evaluate potential targets. Once a network map is created, the team can use it in later phases. For example, they might execute an ARP spoofing attack, which allows them to intercept network traffic more efficiently.

In relation to these activities, this scenario addresses the following point from the thesis proposal:

‘Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen.’

The workshop led to the milestone *NSAK Scenario: Network Discovery – Red Team (must)*, in which each step for discovering and mapping a network is documented and structured for later implementation.

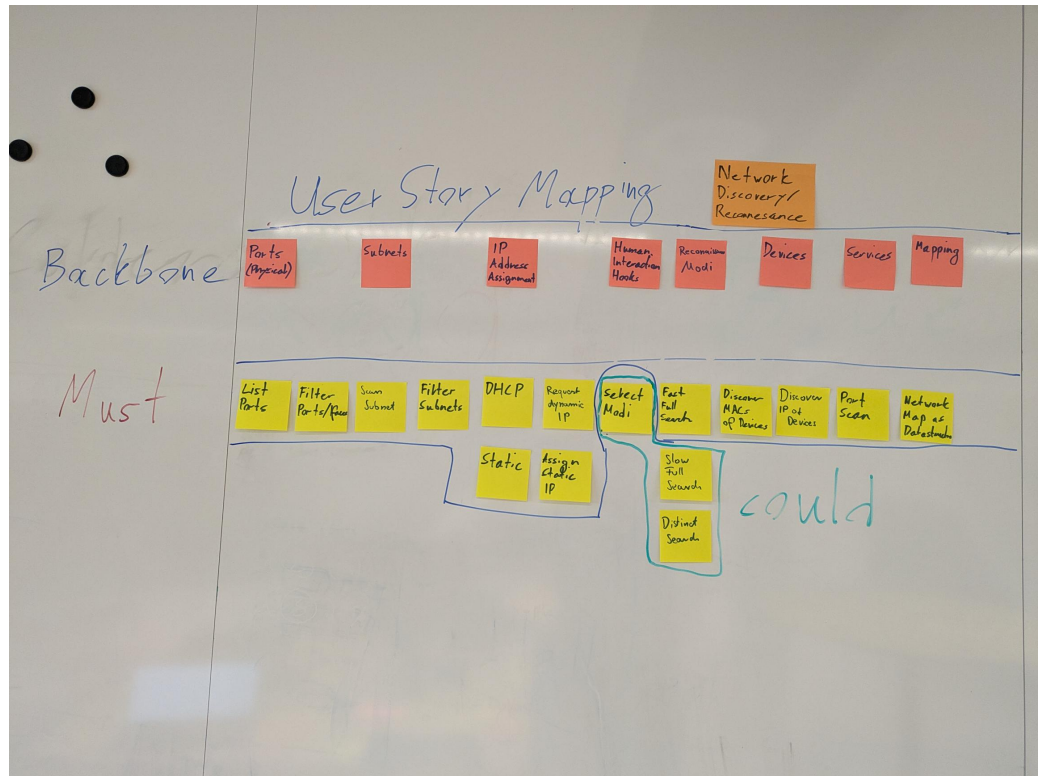


Figure A.12.: NSAK Scenario: Network Discovery - Red Team

A.11.3. NSAK Scenario: TLS Downgrade Poodle - Red Team

TLS Downgrade Poodle Milestone

Objectives

This workshop demonstrates how attackers can exploit TLS downgrade vulnerabilities, such as POODLE, to intercept encrypted communication.

In this scenario, the infamous POODLE attack is implemented, in which a malicious actor tries to convince a client and a server to use the vulnerable TLS version 1.1 instead of 1.2 or 1.3 to encrypt HTTP traffic. This attack requires the attacker to sit between the client and the server (a man-in-the-middle, or MITM), allowing them to intercept and control the traffic. When the client initiates a TLS handshake, the attacker drops packets to the server and sends TCP reset packets to the client, eventually trying older TLS versions. Because TLS 1.1 is vulnerable to a CBC Padding attack, it is possible to create a padding oracle that gradually decrypts the cookie and hijacks the user's session.

Building on this, the scenario directly addresses the following aspect from the thesis proposal:

‘Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen’

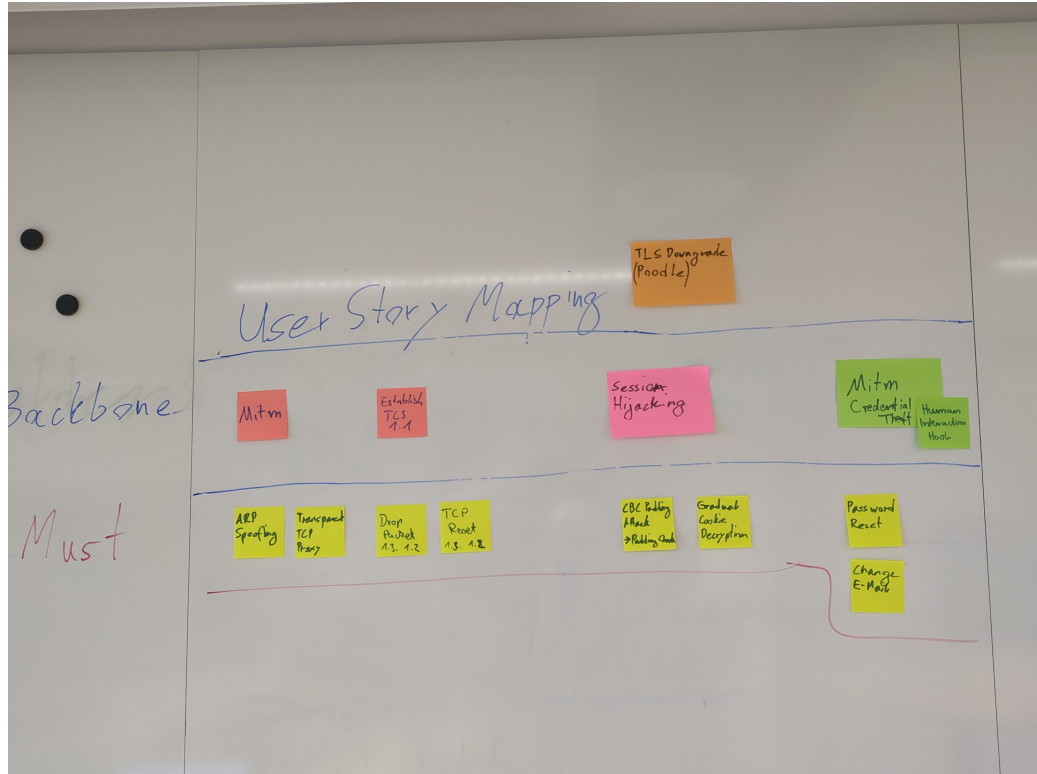


Figure A.13.: NSAK Scenario: TLS Downgrade Poodle - Red Team

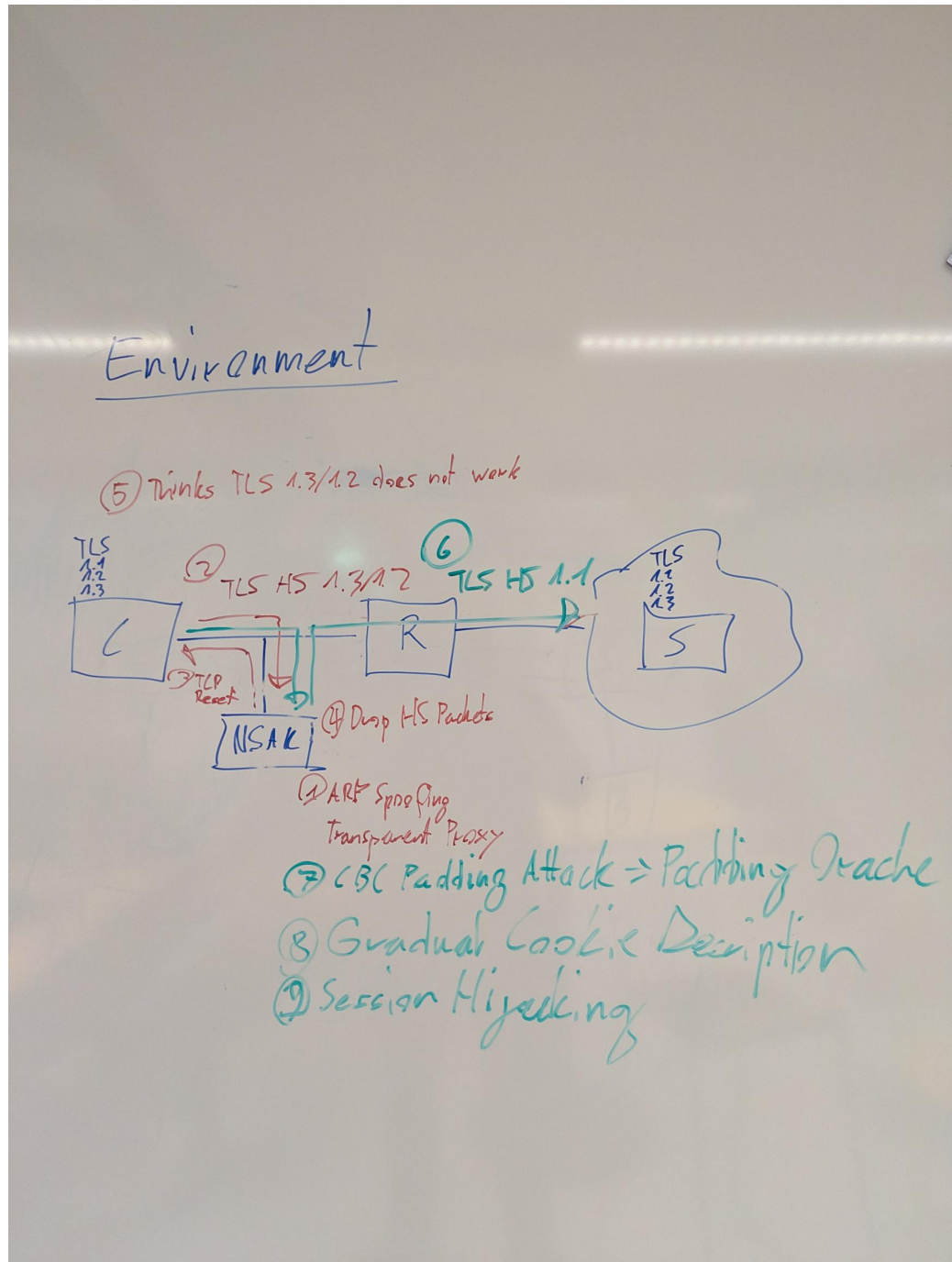


Figure A.14.: NSAK Scenario: TLS Downgrade Poodle -Topology

A.11.4. NSAK Scenario: Honeypot - Blue Team

Honeypot Milestone

Honeyspots can serve as an additional form of intrusion detection and help Blue Teams detect attackers in a network. The system tries to attract malicious actors by providing an attack surface as large as possible; a simple way to achieve this is to open ports. After an attacker tries to connect to any of the open ports, the system notifies an operator and may close all open ports to prevent further attempts to compromise the Honeypot. The operator can then manually intervene to identify and isolate the attacker and analyze the captured payload.

This scenario covers the following bullet point from the thesis proposal:
 ‘Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen’

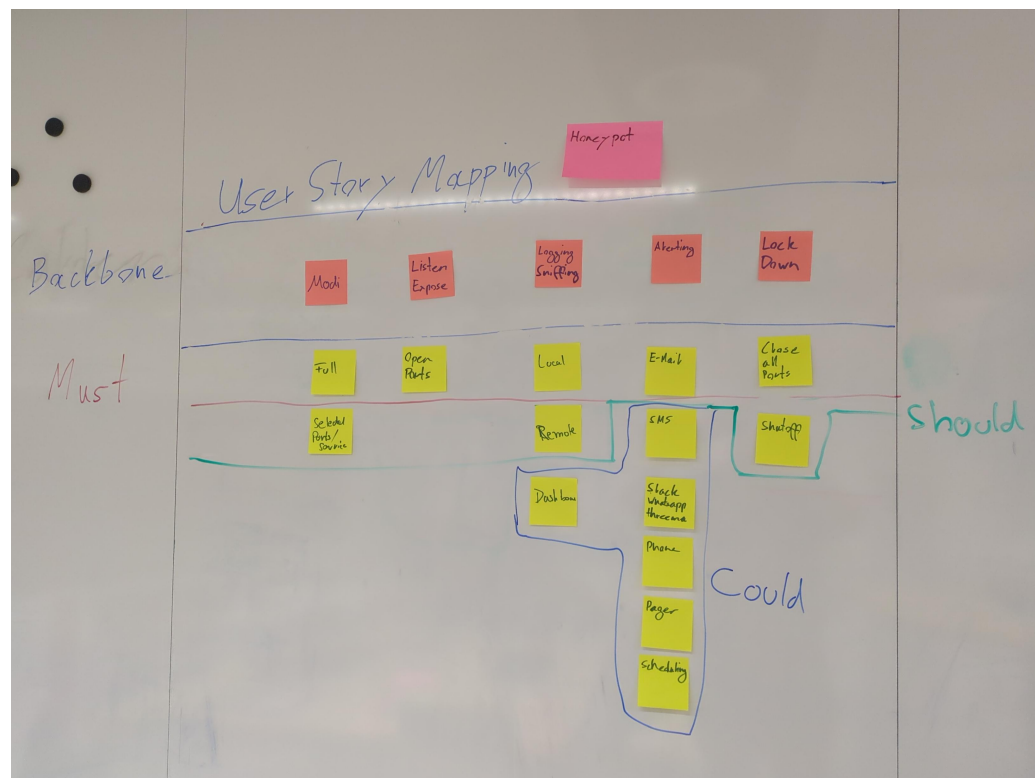


Figure A.15.: NSAK Scenario: Honeypot - Blue Team

A.11.5. NSAK Scenario: Traffic Capture - Red Team

Traffic Capture Milestone

Capturing, exfiltrating, and analyzing traffic from a targeted network is a key Red Team activity and crucial for initial reconnaissance and planning the next phases of the attack. Based on the gathered intelligence, network participants and services can be identified, and possible attack vectors may be evaluated. If unencrypted traffic is present on the network, session data and credentials could already have been extracted.

This scenario addresses the following bullet point from the thesis proposal: ‘Untersuchung und Auswertung der Netzwerkkommunikation zwischen Endgeräten und externen Netzen.’

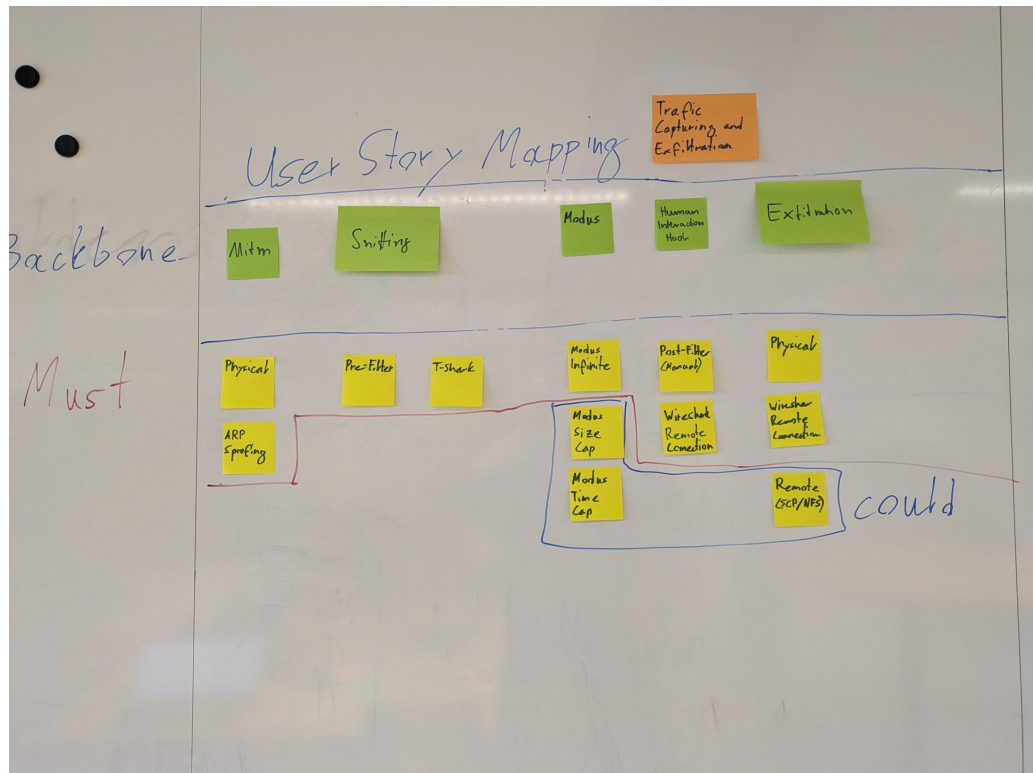


Figure A.16.: NSAK Scenario: Traffic Capture - Red Team

A.11.6. NSAK Scenario: Intrusion Detection - Blue Team

Intrusion Detection Milestone

Intrusion detection is one of the core Blue Team activities and aims to identify malicious actors and misconfigured clients in production network environments. This can be achieved by introducing a system at a traffic mirroring port or as a central point in a network. The intrusion detection system can leverage a wide array of security policies, reference traffic, and other markers to effectively iden-

tify anomalies. As soon as suspicious activity is detected or a policy is violated, automated measures may be initiated, and security engineers will be automatically alerted for manual intervention.

This scenario covers the following bullet point from the thesis proposal: “Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen”

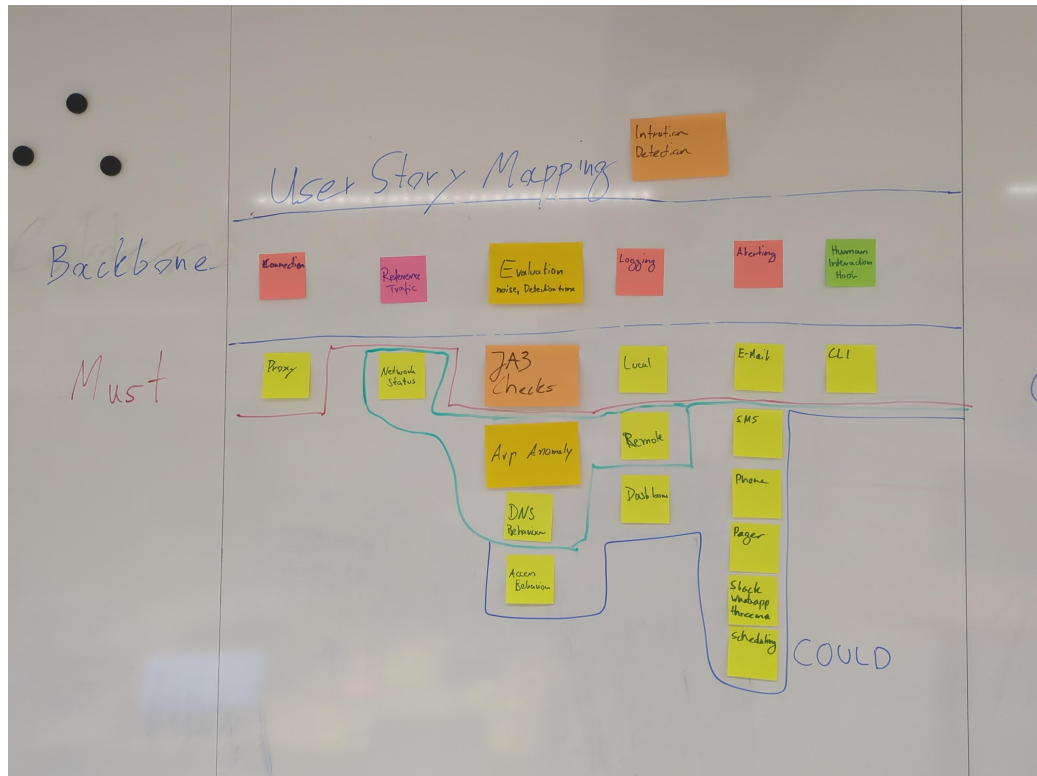


Figure A.17.: NSAK Scenario: Intrusion Detection - Blue Team

A.12. Self-Hosted LLM

During the risk assessment [A.7.9](#) we identified that the AI tokens required to use cloud based LLMs might be very expensive, and it's really hard to estimate the costs. One of the possibilities to avoid this risk was the usage of self-hosted LLMs, and luckily we already had existing hardware at home which is suitable for running smaller LLMs. Besides eliminating the costs for AI tokens, at least during the development phase, it also allows the quick evaluation of different AI Models. Depending on the quality and performance of the smaller models running in the self-hosted setup we can consider not using a cloud based LLM at all.

A.12.1. Hardware Setup

Component	Specification
CPU	AMD Ryzen 5 3600 (6 cores / 12 threads, up to 4.2 GHz)
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
System Memory	32 GB RAM

Table A.15.: Hardware configuration of the self-hosted LLM inference server.

The 8 GB VRAM of the RTX 3070 allows us to run models with up to 7 billion parameters, depending on the specific model [\[10\]](#).

A.12.2. System Setup

The system setup described in [A.1](#) will configure and expose the Ollama API and OpenWeb UI via NGINX, which provided a chat GUI to interact with Ollama. In contrast to Ollama, which runs directly on the host, the other services are run in containers managed by rootless podman and defined the docker-compose.yml [A.2](#). The Nginx default.conf [A.3](#) configures the web server to terminate the SSL connection and act as reverse proxy for the “Open WebUI” and the Ollama API under the directory /llm/.

Services:

- ▶ Ollama: LLM manager, exposing an API for prompts and other interactions (host).
- ▶ Swag: Nginx web server with Certbot for automated SSL certificate generation (containerized).
- ▶ Open WebUI: Web based chat interface for LLM managers (containerized).

```
1 # Login as default user in sudoers or root user
2
3 # Check nvidia driver is loaded and the GPU was correctly detected, otherwise install the
  according drivers
4 nvidia-smi
5
6 # Install Ollama, podman, podman-compose and enable systemd service
7 sudo pacman -Syu ollama ollama-cuda podman podman-compose
8 sudo systemctl enable --now ollama
9 sudo systemctl status ollama
10
11 # Download and run a qwen coder 7b model for testing
12 ollama pull qwen2.5-coder:7b
13 ollama run qwen2.5-coder
14
15 # Create a new user for isolating the services
16 sudo useradd -m ai
17 sudo passwd ai
18
19 # Enable linger for user ai, required to run services such as rootless containers without being
  logged in
20 sudo loginctl enable-linger ai
21
22 # Edit the sysctl.conf file
23 sudo vim /etc/sysctl.conf
24
25 # Add the following lines, required for rootless podman to use the ports 443 and 80
26 net.ipv4.ip_unprivileged_port_start=443
27 net.ipv4.ip_unprivileged_port_start=80
28
29 # Load the new config
30 sudo sysctl -p
31
32 # Login as user ai and create the following directory and file
33 mkdir -p .config/containers
34 vim .config/containers/containers.conf
35
36 # Add the following lines, required for rootless podman (podman manages cgroups directly
  instead of systemd)
37 [engine]
38 cgroup_manager="cgroupfs"
39
40 # Create a docker compose file and add contents from lst:nginx-openwebui-docker-compose.yml
41 vim docker-compose.yml
42
43 # Start the services, so that they create the volume mappings etc, the containers will probably
  fail for now
44 podman-compose up -d
45 podman-compose down
46
47 # Open the nginx default.conf and add the contents from lst:nginx-default-conf
48 vim swag/nginx/site-confs/default.conf
49
50 # Start the services again, this time they should run successfully.
51 # Certbot will fail to perform the http challenge for issuing the TLS certificate,
52 # as long as DynDNS and the port forwarding are not set up properly.
53 podman-compose up -d
```

Listing A.1: Local LLM System Setup

```
1 services:
2   swag:
3     image: lscr.io/linuxserver/swag:latest
4     container_name: swag
5     cap_add:
6       - NET_ADMIN
7     environment:
8       - PUID=1000
9       - PGID=1000
10      - TZ=Europe/Zurich
11      - URL=hiube.ch
12      - VALIDATION=http
13      - SUBDOMAINS=ai,
14      - EMAIL=<my-email>
15      - ONLY_SUBDOMAINS=true
16      - EXTRA_DOMAINS= #optional
17      - STAGING=false #optional
18      - DISABLE_F2B= #optional
19      - SWAG_AUTORELOAD= #optional
20      - SWAG_AUTORELOAD_WATCHLIST= #optional
21     volumes:
22       - /home/ai/swag:/config
23     ports:
24       - 443:443
25       - 80:80 #optional
26       - 443:443/udp #optional
27     restart: unless-stopped
28
29   open-webui:
30     image: ghcr.io/open-webui/open-webui:main
31     container_name: open-webui
32     volumes:
33       - /home/ai/open_webui:/app/backend/data # Use a named volume for persistence
34     environment:
35       OLLAMA_BASE_URL: http://host.docker.internal:11434
36     restart: always
```

Listing A.2: /home/ai/docker-compose.yml

```

1  # redirect all traffic to https
2  server {
3      listen 80 default_server;
4      listen [::]:80 default_server;
5      server_name ai.hiube.ch;
6
7      location / {
8          return 301 https://$host$request_uri;
9      }
10 }
11
12 # main server block
13 server {
14     listen 443 ssl default_server;
15     listen [::]:443 ssl default_server;
16     server_name ai.hiube.ch;
17     include /config/nginx/ssl.conf;
18     root /config/www;
19     index index.html index.htm index.php;
20
21     # enable subfolder method reverse proxy confs
22     # include /config/nginx/proxy-confs/*.subfolder.conf;
23
24     # Reverse proxy for Open WebUI
25     location / {
26         proxy_pass http://open-webui:8080;
27         proxy_set_header Host $host;
28         proxy_set_header X-Real-IP $remote_addr;
29         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
30         proxy_set_header X-Forwarded-Proto $scheme;
31         # Required for WebSockets
32         proxy_http_version 1.1;
33         proxy_set_header Upgrade $http_upgrade;
34         proxy_set_header Connection "upgrade";
35         proxy_read_timeout 900s;
36         proxy_send_timeout 900s;
37     }
38
39     # Reverse proxy for Ollama
40     location /llm/ {
41         auth_basic "Restricted Access";
42         auth_basic_user_file /etc/nginx/.htpasswd;
43
44         # If authentication is successful, forward the request to Ollama
45         # The trailing slash is important, so that ollama does not see the /llm/ as part of the
46         # path!
47         proxy_pass http://host.docker.internal:11434/;
48         proxy_set_header Host $host;
49         proxy_set_header X-Real-IP $remote_addr;
50         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
51
52         # Increase timeouts for slow model responses to prevent 504 Gateway Timeout errors
53         proxy_connect_timeout 300s;
54         proxy_send_timeout 300s;
55         proxy_read_timeout 300s;
56     }
57 }

```

Listing A.3: swag/nginx/site-confs/default.conf

System Hardening

It is recommended to put a proper firewall before the services we want to expose to the internet, but for our purposes we can just disable the port forwarding when we don't use the AI. It still makes sense to perform a minimal system hardening as shown in the following listing [A.4](#), before exposing the services to the internet.

```
1  # Install UFW
2  sudo pacman -Syu ufw
3  sudo ufw status
4
5  # Allow all outgoing connections
6  sudo ufw default allow outgoing
7
8  # Deny all incoming connections per default
9  sudo ufw default deny incoming
10
11 # Allow SSH for remote access
12 sudo ufw allow ssh
13
14 # Allow HTTP for certbot HTTP challenge
15 sudo ufw allow http
16
17 # Allow HTTPS for Open WebUI
18 sudo ufw allow https
19
20 # Allow default Ollama port
21 sudo ufw allow 11434/tcp
22
23 # Make sure to run this command after adding the "allow" rules!
24 sudo ufw enable
25
26 sudo ufw status
```

Listing A.4: UFW configuration

Expose Open WebUI and Ollama

For exposing the services from the home network via HTTPS to the internet, we need a domain name which points to the home router. To achieve this we can use DynDNS, which is provided for free by Infomaniak where the domain “hiube.ch” was registered, and is supported by the FRITZ!Box used in the home network. The screenshots [A.19](#) and [A.18](#) are showing the respective configurations.

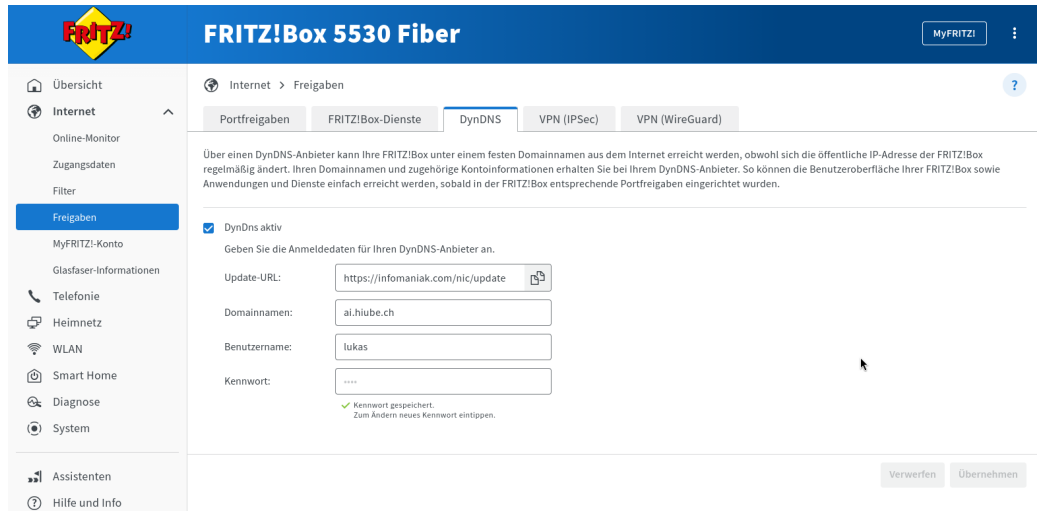


Figure A.18.: FRITZ!Box DynDNS Setup

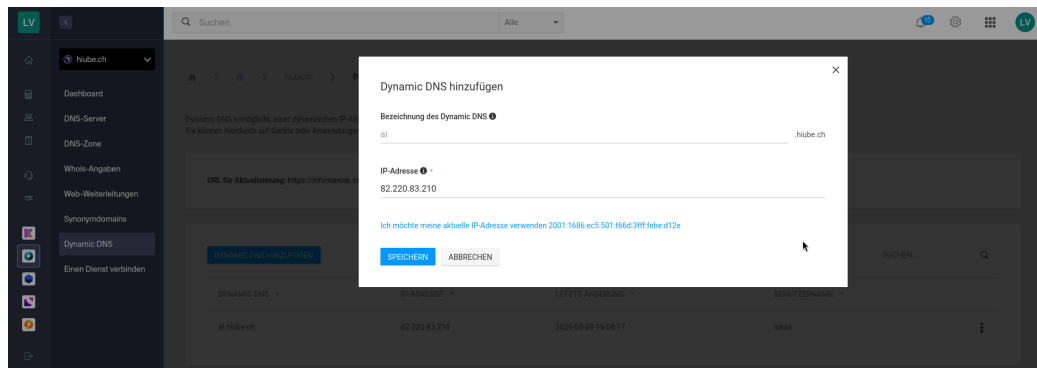


Figure A.19.: Infomaniak DynDNS Setup

Now that the domain name “ai.hiube.ch” points to the router in the home network, we need to redirect the HTTP(S) traffic to the LLM server. This can be achieved with port forwarding rules on the FRITZ!Box, as shown in the screenshot [A.20](#).

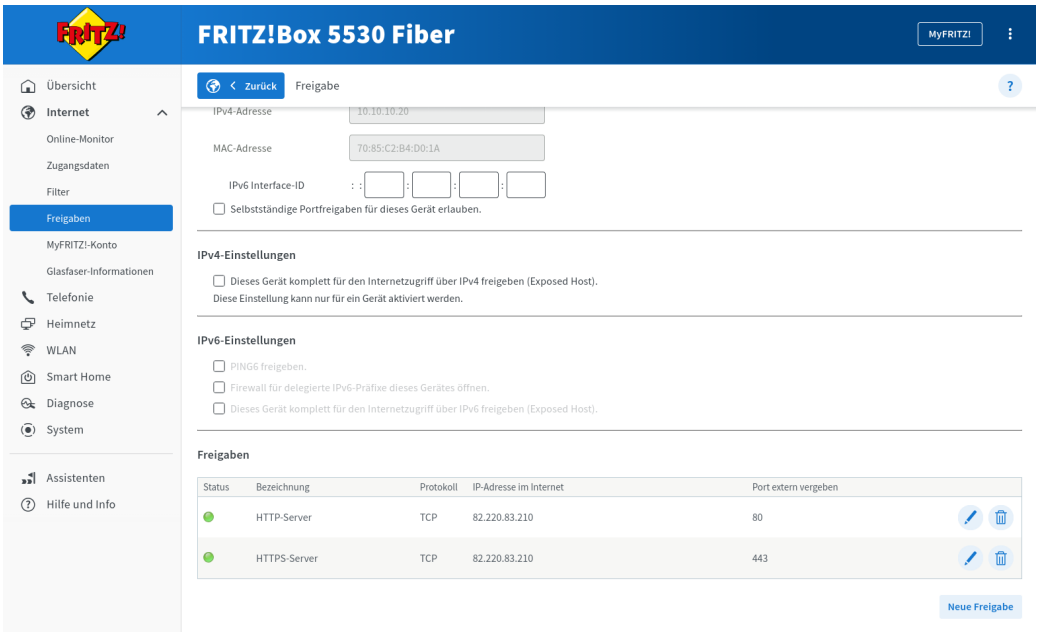


Figure A.20.: FRITZ!Box DynDNS Setup

The Open WebUI and Ollama API should already be reachable via “ai.hiube.ch”, but we will receive a security warning because there is no valid SSL certificate. The easiest way to generate the certificate, is to trigger certbot by restarting the “swag” service as shown in the listing below [A.5](#).

```
1 # Login as user ai and restart the swag service
2 podman-compose restart swag
```

Listing A.5: Restart SWAG Container

A.12.3. Evaluation

For quickly evaluate models and assess their capabilities we can now use the Open WebUI served under “ai.hiube.ch”. The screenshot [A.21](#) shows a quick test with “qwen2.5-coder:7B”.

A. Project Management

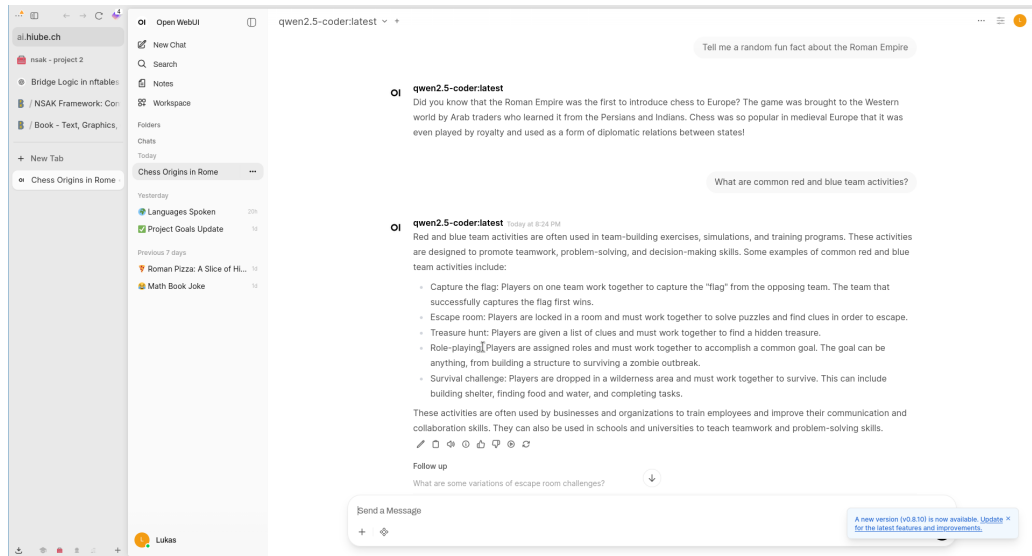


Figure A.21.: FRITZ!Box DynDNS Setup

Later on we can use the Ollama API served under `/llm/` as a backend for AI agents.